MyCodeSchool

Exerciții — algoritmi pe matrici

Recapitulare · Victor · C++ · 33 de algoritmi, 52 de probleme

`algoritmi.h` **e doar de studiu.** Are înăuntru algoritmii de mai jos, gata scriși. Îl citești, îl închizi, apoi **scrii tu** algoritmul în headerul tău. Dacă te blochezi, îl trasezi pe hârtie, nu-l copiezi.

Cum lucrezi la fișă:

- **pentru fiecare secțiune îți faci headerul ei** — numele e scris în titlul secțiunii;
- sus în header scrii **tu** algoritmii de la „Studiezi”, apoi funcțiile care îi folosesc;
- garda se scrie fără cratimă: `PARCURGERE_H_INCLUDED`, nu `PARCURGERE-H_INCLUDED`;
- proprietățile simple de care ai nevoie (prim, palindrom, oglindit…) le scrii tu, în funcții ajutătoare;
- liniile și coloanele se numerotează de la 0; matricea vine din `matrice.in`, valorile date (k, n, m…) se citesc de la tastatură;
- headerele noi le adaugi în proiectul `matrici` din Visual Studio; în `matrici.cpp` incluzi doar headerul la care lucrezi (citirea din `matrice.in` și afișarea le scrii tu în el); rulezi cu **Ctrl+F5**.

Reguli:

- **Nu ai voie** să incluzi `algoritmi.h`, să chemi ceva din el sau să copiezi din el.
- **Ai voie** să chemi funcțiile **tale**, din headerele tale, oricâte, în orice combinație.
- **Nu ai voie:** `#include <algorithm>`, `sort`, `vector` din bibliotecă.

1. Parcurgere pe linii și coloane · `parcurgere.h`

Studiezi din `algoritmi.h`: `sumaLinie, sumaColoana, pozMinim, pozMaxim, frecventaElement, linieCrescatoare, linieDescrescatoare, coloanaCrescatoare, coloanaDescrescatoare`

Linia cu suma maximă — 1.1

Scrie o funcție care afișează linia cu cea mai mare sumă a elementelor și suma ei. La egalitate, prima linie.

```
1 2 3      ->  linia 1 , suma 18
4 6 8
5 7 2

1 2      ->  linia 0 , suma 3
3 0
```

Coloane ordonate — 1.2

Scrie o funcție care întoarce câte coloane au elementele ordonate, crescător sau descrescător. O coloană constantă se numără o singură dată.

```
1 9 5      ->  2
2 7 5
3 8 5

3 4      ->  1
2 5
1 4

4 1 7      ->  3
```

Cât de departe sunt

1.3

Scrie o funcție care întoarce cât de departe sunt minimul și maximul: diferența dintre linii plus diferența dintre coloane, fiecare fără semn. Dacă apar de mai multe ori, se iau primele apariții.

```
3 8      ->  2
5 1
9 4

1 2 3      ->  4
4 5 6
7 8 9
```

Cel mai frecvent element

1.4

Scrie o funcție care afișează elementul care apare de cele mai multe ori în matrice. La egalitate, cel mai mic.

```
3 7 3      ->  3
5 7 3

1 2      ->  1
2 1
```

Linie crescătoare, coloană descrescătoare

1.5

Scrie o funcție care afișează prima linie ordonată crescător și ultima dintre coloanele ordonate descrescător. Dacă una dintre ele nu există, afișează NU EXISTA.

```
8 3 6      ->  linia 1 , coloana 2
1 2 5
4 2 1

1 9 4      ->  linia 2 , coloana 1
3 7 6
2 5 8

1 2      ->  NU EXISTA
3 4
```

2. Diagonale, triunghiuri și zone N/S/E/V · `diagonale.h`

Studiezi din `algoritmi.h`: `sumaDiagonalaPrincipala, sumaDiagonalaSecundara, sumaDeasupraDiagonaleiPrincipale, sumaSubDiagonalaPrincipala, sumaDeasupraDiagonaleiSecundare, sumaSubDiagonalaSecundara, sumaNord, sumaSud, sumaEst, sumaVest (matrice pătratică)`

Cele patru triunghiuri

2.1

Scrie o funcție care afișează, pe un rând, sumele elementelor aflate: deasupra diagonalei principale, sub diagonala principală, deasupra diagonalei secundare, sub diagonala secundară, în ordinea asta.

```
1 2 3      ->  11 19 7 23
4 5 6
7 8 9

1 2      ->  2 3 1 4
3 4

5      ->  0 0 0 0
```

Diagonala mai grea

2.2

Scrie o funcție care afișează care diagonală are suma mai mare: principala, secundara, sau egale.

```
1 2 3      ->  egale
4 5 6
7 8 9

5 1      ->  principala
2 4

1 9      ->  secundara
8 2
```

Peste media triunghiului

2.3

Scrie o funcție care întoarce câte elemente de pe diagonala principală sunt strict mai mari decât media aritmetică (ca număr real) a elementelor de deasupra ei. Dacă deasupra diagonalei nu e niciun element, afișează NU EXISTA.

```
1 2 3      ->  2
4 5 6
7 8 9

5 9 1      ->  0
2 5 8
7 3 5

7      ->  NU EXISTA
```

Zona cu suma maximă

2.4

Zonele fără diagonale: Nord (deasupra ambelor diagonale), Sud (sub ambele), Est (la dreapta), Vest (la stânga). Scrie o funcție care afișează zona cu suma cea mai mare și suma ei. La egalitate, în ordinea Nord, Sud, Est, Vest.

```
 1  2  3  4  5      ->  Sud 87
 6  7  8  9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25

1 2 3      ->  Sud 8
4 5 6
7 8 9

1 1 1      ->  Nord 1
1 1 1
1 1 1
```

Zone în echilibru

2.5

Scrie o funcție care afișează DA dacă Nord și Sud au aceeași sumă și Est și Vest au aceeași sumă, altfel NU.

```
9 2 9      ->  DA
4 5 4
9 2 9

1 2 3      ->  NU
4 5 6
7 8 9

1 2      ->  DA
3 4
```

3. Chenare și spirala · `chenare.h`

Studiezi din `algoritmi.h`: `nrChenare, nrElementeChenar, afisareChenar, sumaChenar, maximChenar, algoritmSpirala`

Câte elemente are fiecare chenar

3.1

Se citesc n și m. Scrie o funcție care afișează câte elemente are fiecare chenar al unei matrici cu n linii și m coloane, de la gradul 0, câte unul pe linie. Matricea nu se citește.

```
n = 5 , m = 3    ->  Chenar 0: 12
                     Chenar 1: 3

n = 4 , m = 4    ->  Chenar 0: 12
                     Chenar 1: 4

n = 1 , m = 4    ->  Chenar 0: 4
```

Suma chenarului de grad k

3.2

Se citește k. Scrie o funcție care afișează suma elementelor chenarului de grad k, sau NU EXISTA dacă matricea nu are un astfel de chenar.

```
1 2 3      k = 0  ->  40
4 5 6      k = 1  ->  5
7 8 9      k = 2  ->  NU EXISTA

 1  2  3      k = 0  ->  96
 4  5  6      k = 1  ->  24
 7  8  9
10 11 12
13 14 15
```

Chenarul cel mai greu

3.3

Scrie o funcție care afișează gradul chenarului cu suma cea mai mare, suma lui și maximul de pe el. La egalitate, chenarul cu gradul mai mic.

```
1 1 1 1      ->  Chenar 0, suma 12, maxim 1
1 3 3 1
1 3 3 1
1 1 1 1

1 1 1      ->  Chenar 1, suma 9, maxim 9
1 9 1
1 1 1
```

Spirala pe dos

3.4

Scrie o funcție care afișează elementele în spirală începând din centru: chenarele de la ultimul la cel exterior, fiecare în ordinea lui obișnuită.

```
1  2  3  4      ->  6 7 1 2 3 4 8 12 11 10 9 5
5  6  7  8
9 10 11 12

1 2 3 4      ->  1 2 3 4

1      ->  1 2 3
2
3
```

Matricea spirală

3.5

Se citesc n și m. Scrie o funcție care construiește o matrice cu n linii și m coloane, cu valorile 1, 2, 3, … puse în spirală, și o afișează.

```
n = 3 , m = 4    ->   1  2  3 4
                     10 11 12 5
                      9  8  7 6

n = 4 , m = 1    ->  1
                     2
                     3
                     4

n = 1 , m = 3    ->  1 2 3
```

4. Ștergeri · `stergeri.h`

Studiezi din `algoritmi.h`: `stergereLinie, stergereColoana`

Linia și coloana minimului 4.1

Scrie o funcție care șterge linia și coloana pe care se află minimul (prima apariție) și afișează matricea rămasă. Dacă nu rămâne niciun element, afișează MATRICE VIDA.

```
4 9 2    ->  4 2
7 1 5        8 6
8 3 6

1 5      ->  2
7 2

3      ->  MATRICE VIDA
```

Liniile neordonate 4.2

Scrie o funcție care șterge toate liniile care nu au elementele ordonate (nici crescător, nici descrescător). Dacă nu rămâne nicio linie, afișează MATRICE VIDA.

```
1 2 3    ->  1 2 3
5 1 4        9 7 2
2 8 3
9 7 2

1 3 2      ->  MATRICE VIDA
```

Al doilea număr prim 4.3

Scrie o funcție care șterge linia și coloana pe care se află al doilea număr prim din matrice (parcurgere pe linii). Dacă matricea nu are cel puțin două numere prime, afișează NU EXISTA.

```
 4  7  9    ->   7  9
11  6  8        12 13
10 12 13

4 6      ->  NU EXISTA
9 7
```

Coloane cu palindrom de 3 cifre 4.4

Scrie o funcție care șterge toate coloanele care conțin cel puțin un număr palindrom cu exact 3 cifre. Dacă nu rămâne nicio coloană, afișează MATRICE VIDA.

```
121  45  78  9    ->   78  9
 12 232  78 33         78 33
 15  45 100  7        100  7

101 202      ->  MATRICE VIDA

11 22    ->  11 22
33 44        33 44
```

Elementul cel mai frecvent 4.5

Scrie o funcție care șterge linia și coloana elementului cu frecvența maximă. La egalitate, se alege cel mai mic; se ia prima lui apariție.

```
3 7 3      ->  7 3
5 7 3

1 2      ->  1
2 1
```

5. Inserări · `inserari.h`

Studiezi din `algoritmi.h`: `inserareLinie, inserareColoana  (doar fac loc; completezi tu)`

Zero între coloane 5.1

Scrie o funcție care inserează între oricare două coloane vecine o coloană cu zerouri.

```
1 2 3    ->  1 0 2 0 3
4 5 6        4 0 5 0 6

7    ->  7
8        8
```

Dublul liniei grele 5.2

Scrie o funcție care inserează, după linia cu suma maximă (prima, la egalitate), dublul ei.

```
1 2 3    ->   1  2  3
9 8 7          9  8  7
5 7 9         18 16 14
               5  7  9

2 4    ->  2 4
3 3        4 8
           3 3

1 3    ->  1  3
2 9        2  9
           4 18
```

Jumătățile liniilor pare 5.3

Scrie o funcție care inserează, după fiecare linie cu toate elementele pare, o linie formată din jumătățile elementelor ei.

```
12 48 60    ->  12 48 60
15 22 31          6 24 30
84 26 70         15 22 31
                 84 26 70
                 42 13 35

2 4    ->  2 4
6 8        1 2
           6 8
           3 4
```

Oglinditul înaintea coloanei ușoare

5.4

Scrie o funcție care inserează, înaintea coloanei cu suma minimă (prima, la egalitate), o coloană formată din oglinditul fiecărui element al ei.

```
12 45 70    ->  21 12 45 70
31 22 13        13 31 22 13
24 10 56        42 24 10 56

10 4    ->  1 10 4
 2 8        2  2 8
```

Divizori sub ultima linie descrescătoare

5.5

Scrie o funcție care inserează, după ultima linie ordonată descrescător, o linie în care fiecare element este numărul de divizori proprii ai elementului de deasupra lui (divizorii diferiți de 1 și de numărul însuși). Dacă nu există o astfel de linie, afișează NU EXISTA.

```
30 20 10    ->  30 20 10
 5 12 40         5 12 40
60 18  6        60 18  6
12 12 15        10  4  2
                12 12 15

1 2      ->  NU EXISTA
3 4

3 1    ->  3 1
9 4        9 4
           1 1
```

6. Interschimbări și sortări · `interschimbari.h`

Studiezi din `algoritmi.h`: `interschimbareLinii, interschimbareColoane, sortareCrescLinie, sortareLiniiDupaColoana`

Liniile în oglindă

6.1

Scrie o funcție care interschimbă prima linie cu ultima, a doua cu penultima, și tot așa. Linia din mijloc, dacă există, rămâne pe loc.

```
1 2    ->  7 8
3 4        5 6
5 6        3 4
7 8        1 2

1 2    ->  5 6
3 4        3 4
5 6        1 2
```

Coloana grea pe primul loc

6.2

Scrie o funcție care interschimbă coloana cu suma maximă (prima, la egalitate) cu prima coloană.

```
1 9 2    ->  9 1 2
3 8 4        8 3 4
5 7 6        7 5 6

1 2    ->  1 2
2 1        2 1
```

Minim cu maxim

6.3

Scrie o funcție care interschimbă linia minimului cu linia maximului (primele apariții). Dacă sunt pe aceeași linie, afișează Nu putem interschimba!

```
4 9 2    ->  7 1 5
7 1 5        4 9 2
8 3 6        8 3 6

5 2 8    ->  2 9 7
2 9 7        5 2 8
3 4 6        3 4 6

1 9      ->  Nu putem interschimba!
3 4
```

Liniile neordonate, sortate

6.4

Scrie o funcție care ordonează crescător fiecare linie care nu este ordonată (nici crescător, nici descrescător). Celelalte linii rămân neatinse.

```
3 1 2    ->  1 2 3
1 2 3        1 2 3
9 5 1        9 5 1
4 7 5        4 5 7

2 1 3    ->  1 2 3
7 7 7        7 7 7
```

Liniile după coloana grea

6.5

Scrie o funcție care ordonează liniile crescător după valorile din coloana cu suma maximă (prima, la egalitate). Liniile cu aceeași valoare în acea coloană rămân în ordinea inițială.

```
5 1 2    ->  5 1 2
3 9 9        8 4 4
8 4 4        1 7 0
1 7 0        3 9 9

2 7    ->  2 7
1 7        1 7
3 7        3 7
```

7. Generare de matrici · `generare.h`

Pentru fiecare figură scrie o funcție care generează matricea **pentru orice n** și o afișează cu `afisare`. Figurile sunt desenate pentru n = 6 (și n = 5 unde formula se vede abia la două dimensiuni).

Partea 1

Figura 1 — 7.1

```
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
```

Figura 2 — 7.2

```
1  2  3  4  5  6
2  4  6  8 10 12
3  6  9 12 15 18
4  8 12 16 20 24
5 10 15 20 25 30
6 12 18 24 30 36
```

Figura 3 — 7.3

```
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
```

Figura 4 — 7.4

```
 1  2  3  4  5  6
 7  8  9 10 11 12
13 14 15 16 17 18
19 20 21 22 23 24
25 26 27 28 29 30
31 32 33 34 35 36
```

Figura 5 — 7.5

```
1  7 13 19 25 31
2  8 14 20 26 32
3  9 15 21 27 33
4 10 16 22 28 34
5 11 17 23 29 35
6 12 18 24 30 36
```

Partea 2

Figura 6 — 7.6

```
1 1 1 1 1 1
1 2 2 2 2 2
1 2 3 3 3 3
1 2 3 4 4 4
1 2 3 4 5 5
1 2 3 4 5 6
```

Figura 7 — 7.7

```
1 2 3 4 5 6
2 2 3 4 5 6
3 3 3 4 5 6
4 4 4 4 5 6
5 5 5 5 5 6
6 6 6 6 6 6
```

Figura 8 — 7.8

```
0 1 2 3 4 5
1 0 1 2 3 4
2 1 0 1 2 3
3 2 1 0 1 2
4 3 2 1 0 1
5 4 3 2 1 0
```

Figura 9 — 7.9

```
1 2 3 4  5  6
2 3 4 5  6  7
3 4 5 6  7  8
4 5 6 7  8  9
5 6 7 8  9 10
6 7 8 9 10 11
```

Figura 10 — 7.10

```
1 2 3 4 5 6
2 3 4 5 6 1
3 4 5 6 1 2
4 5 6 1 2 3
5 6 1 2 3 4
6 1 2 3 4 5
```

Partea 3

Figura 11 — 7.11

```
1 2 3 4 5 0
2 4 0 2 4 0
3 0 3 0 3 0
4 2 0 4 2 0
5 4 3 2 1 0
0 0 0 0 0 0
```

Figura 12 — 7.12

```
1 1 1 1 1 1
1 2 2 2 2 1
1 2 3 3 2 1
1 2 3 3 2 1
1 2 2 2 2 1
1 1 1 1 1 1
```

Figura 13 — 7.13

```
 1  2  3  4  5  6
12 11 10  9  8  7
13 14 15 16 17 18
24 23 22 21 20 19
25 26 27 28 29 30
36 35 34 33 32 31
```

Figura 14 — 7.14

```
0 1 1 1 1 0
4 0 1 1 0 2
4 4 0 0 2 2
4 4 0 0 2 2
4 0 3 3 0 2
0 3 3 3 3 0
```

Figura 15 — 7.15

```
 1  2  3  4  5  6
20  0  0  0  0  7
19  0  0  0  0  8
18  0  0  0  0  9
17  0  0  0  0 10
16 15 14 13 12 11
```

Provocări

Provocarea 1 — 7.16

```
n = 6             n = 5
1 0 0 0 0 0       1 0 0 0 0
2 1 0 0 0 0       2 1 0 0 0
3 1 1 0 0 0       3 1 1 0 0
4 2 1 1 0 0       4 2 1 1 0
5 2 1 1 1 0       5 2 1 1 1
6 3 2 1 1 1
```

Provocarea 2 — 7.17

```
1 1 1 1 1 1
0 1 0 1 0 1
0 0 1 0 0 1
0 0 0 1 0 0
0 0 0 0 1 0
0 0 0 0 0 1
```

Provocarea 3 — 7.18

```
 0  0  0  0  0  0
 0  0  0  0  0 12
 0  0  0  0 15 18
 0  0  0 16 20 24
 0  0 15 20 25 30
 0 12 18 24 30 36
```

Provocarea 4 — 7.19

```
n = 6                  n = 5
1 12 13 24 25 36       1 10 11 20 21
2 11 14 23 26 35       2  9 12 19 22
3 10 15 22 27 34       3  8 13 18 23
4  9 16 21 28 33       4  7 14 17 24
5  8 17 20 29 32       5  6 15 16 25
6  7 18 19 30 31
```

Provocarea 5 — 7.20

```
n = 6                    n = 5
 1  0  0  0  0  0         1  0  0  0  0
 2  3  0  0  0  0         2  3  0  0  0
 4  5  6  0  0  0         4  5  6  0  0
 7  8  9 10  0  0         7  8  9 10  0
11 12 13 14 15  0        11 12 13 14 15
16 17 18 19 20 21
```

Provocarea 6 — 7.21

```
1 0  0  0 0 0
1 1  0  0 0 0
1 2  1  0 0 0
1 3  3  1 0 0
1 4  6  4 1 0
1 5 10 10 5 1
```

Provocarea 7 — 7.22

```
n = 6              n = 5
1 1 1 1 1 1        1 1 1 1 1
1 2 1 2 1 2        1 2 1 2 1
1 1 3 1 1 3        1 1 3 1 1
1 2 1 4 1 2        1 2 1 4 1
1 1 1 1 5 1        1 1 1 1 5
1 2 3 2 1 6
```

Pentru fiecare funcție pe care o scrii, verific-o pe:

1. o matrice cu o **singură linie** și una cu o **singură coloană**;
2. o matrice în care **toate** elementele sunt egale;
3. cazul în care răspunsul e pe **prima** linie/coloană, și cel în care e pe **ultima**;
4. cazul în care ce cauți **nu există**.